D20A / 37

Lab - 1: **Experiment No: 1**

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
   - Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
   - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
   - Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
   - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

Program #1: Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib                                                    #error
statement


def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()                         #error
statement
    # Return the hash string
    return hash_string


# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)                                        #error
statement
```

Output:

```
Enter a string: prince

Hash: 00bbf5ef9b6831e986ba5b41481e02a7bd006719d6b044d64155ace664214ce1
```

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce                                    #
Error

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)                                          #
Error
```

Output:

```
Enter a string: Prince
Enter the nonce: 55
Hash Code:
90d4f2d07c2e161879ccd325c6cf0495e5e55015a8d83ca980684d8ce9f29182
```

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
#Error
```

```
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
#Error
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "Prince"
num_zeros = 4

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
#Error
```

Output:

```
Hash: 000068f4cd18342e160e87433c707e39e55bb779312d36b0bfc3201d71e852df
Input String: Prince
Leading Zeros: 4
Expected Nonce: 57748
```

Program 4: Generating Merkle Tree for given set of Transactions

```
import hashlib

def build_merkle_tree(transactions):                    # Error 1

    if len(transactions) == 0:
```

```
    return None

# Step 1: Hash all transactions before combining
transactions = [
    hashlib.sha256(transaction.encode('utf-8')).hexdigest()
    for transaction in transactions
]

print("Initial Transaction Hashes:")
for i, transaction_hash in enumerate(transactions):
    print(f"T{i+1}: {transaction_hash}")

print("\nMerkle Tree Construction:")

level = 1

# Recursive construction of the Merkle Tree
while len(transactions) > 1:

    # If odd number of hashes, duplicate the last hash
    if len(transactions) % 2 != 0:
        transactions.append(transactions[-1])

    new_transactions = []

    for i in range(0, len(transactions), 2):

        # Combine two hashes
        combined = transactions[i] + transactions[i + 1]

        # Hash the combined value
        hash_combined = hashlib.sha256(
            combined.encode('utf-8')
        ).hexdigest()

        new_transactions.append(hash_combined)              # Error 2

        print(
            f"Hash {i//2 + 1}: "
            f"{hash_combined}"
```

```
            )

        transactions = new_transactions

        print()

        level += 1

    return transactions[0]


# Example usage
transactions = [
    "Alice -> Bob : $200",
    "Bob -> Dave : $500",
    "Dave -> Eve : $100",
    "Eve -> Alice : $300",
    "Roo -> Bob : $50"
]

merkle_root = build_merkle_tree(transactions)

print("Merkle Root:", merkle_root)                                  # Error 3
```

Output:

```
Initial Transaction Hashes:
T1: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
T2: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
T3: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
T4: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
T5: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

Merkle Tree Construction:
Hash 1: 471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9
Hash 2: a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
Hash 3: 314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

Hash 1: ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
```

```
Hash 2: 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

Hash 1: 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Merkle Root:
96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0
```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.